

Rapport TP1

Requêtes et base de connaissances

Introduction

1. Oui, d'après ce qu'on lit, Vincent boit de la vodka.
2. On tape `boit(vincent, vodka)`. et on obtient `true`.
3. D'après le fichier, Otto ne boit pas de vodka.
4. On tape `boit(otto, vodka)` et on obtient `false`.
5. En ajoutant la formule `boit(otto, vodka)` et en la retapant, on obtient `true`.
6. En tapant `boit(X, eau)`., on obtient le nom de tous ceux qui ont bu de l'eau.
7. En tapant `boit(simon, X)`., on obtient la liste de tout ce qu'a bu Simon.
A noter que cette commande ne reconnait pas les doublons et on obtient deux fois `cafe`.
8. L'implication est :

$$\forall X, Y \\ boit(X, Y) \rightarrow personne(X)$$

La formule est : `personne(X) :- boit(X, _)`.

La famille

1. `mere(X, Y)` signifie « *X* est la mère de *Y* ». *X* est le mère de *Y* si elle est son parent et que c'est une femme.
Son implication est :

$$\forall X, Y \\ parent(X, Y) \wedge femme(X) \rightarrow mere(X, Y)$$

Le code prolog est :

`mere(X, Y) :- parent(X, Y), femme(X).`

`pere(X, Y)` signifie « *X* est le père de *Y* ».

X est le père de *Y* s'il est son parent et que c'est un homme.

Son implication est :

$$\forall X, Y \\ parent(X, Y) \wedge homme(X) \rightarrow pere(X, Y)$$

Le code prolog est :

`pere(X, Y) :- parent(X, Y), homme(X).`

2. **fils(X, Y)** signifie « X est le fils de Y ».
 X est le fils de Y si c'est un homme et que Y est son parent.
 Son implication est :

$$\forall X, Y \\ parent(Y, X) \wedge homme(X) \rightarrow fils(X, Y)$$

Le code prolog est :

fils(X, Y) :- parent(Y, X), homme(X).

fille(X, Y) signifie « X est la fille de Y ».
 X est la fille de Y si c'est une femme et que Y est son parent.
 Son implication est :

$$\forall X, Y \\ parent(Y, X) \wedge femme(X) \rightarrow fille(X, Y)$$

Le code prolog est :

fille(X, Y) :- parent(Y, X), femme(X).

3. **grandpere(X, Y)** signifie « X est le grand-père de Y ».
 X est le grand-père de Y s'il existe une personne Z dont il est le parent et qui lui est même parent de Y et que X est un homme.
 Son implication est :

$$\forall X, Y, Z \\ parent(X, Z) \wedge parent(Z, Y) \wedge homme(X) \rightarrow grandpere(X, Y)$$

Le code prolog est :

grandpere(X, Y) :- parent(X, Z), parent(Z, Y), homme(X).

grandmere(X, Y) signifie « X est la grand-mère de Y ».
 X est la grand-mère de Y s'il existe une personne Z dont il est le parent et qui lui est même parent de Y et que X est une femme.
 Son implication est :

$$\forall X, Y, Z \\ parent(X, Z) \wedge parent(Z, Y) \wedge femme(X) \rightarrow grandmere(X, Y)$$

Le code prolog est :

grandmere(X, Y) :- parent(X, Z), parent(Z, Y), femme(X).

4. **soeur(X, Y)** signifie « X est la soeur de Y ».
 X est la soeur de Y si X est une femme et s'il existe une personne Z qui est parent à la fois de X et Y .
 Son implication est :

$$\forall X, Y, Z \\ (X \neq Y) \wedge parent(Z, X) \wedge parent(Z, Y) \wedge femme(X) \rightarrow soeur(X, Y)$$

Le code prolog est :

```
soeur(X, Y) :- parent(Z, X), parent(Z, Y), X \== Y, femme(X).
```

frere(X,Y) signifie « X est le frère de Y ».

X est le frère de Y si X est un homme et s'il existe une personne Z qui est parent à la fois de X et Y .

Son implication est :

$$\forall X, Y, Z \\ (X \neq Y) \wedge \text{parent}(Z, X) \wedge \text{parent}(Z, Y) \wedge \text{homme}(X) \rightarrow \text{frere}(X, Y)$$

Le code prolog est :

```
frere(X, Y) :- parent(Z, X), parent(Z, Y), X \== Y, homme(X).
```

5. cousin(X,Y) signifie « X est le cousin de Y ».

X est le cousin de Y si X est un homme et s'il existe une personne Z qui est son parent et qu'elle a un frère ou une soeur T qui est le parent de Y .

Son implication est :

$$\forall X, Y, Z, T \\ (X \neq Y) \wedge \text{parent}(Z, X) \wedge \text{parent}(T, Y) \wedge (\text{frere}(Z, T) \vee \text{soeur}(Z, T)) \wedge \text{homme}(X) \rightarrow \text{cousin}(X, Y)$$

Le code prolog est :

```
cousin(X, Y) :- parent(Z, X), frere(Z, T), parent(T, Y), X \== Y, homme(X).
cousin(X, Y) :- parent(Z, X), soeur(Z, T), parent(T, Y), X \== Y, homme(X).
```

cousine(X,Y) signifie « X est la cousine de Y ».

X est la cousine de Y si X est une femme et s'il existe une personne Z qui est son parent et qu'elle a un frère ou une soeur T qui est le parent de Y .

Son implication est :

$$\forall X, Y, Z, T \\ (X \neq Y) \wedge \text{parent}(Z, X) \wedge \text{parent}(Z, Y) \wedge (\text{frere}(Z, T) \vee \text{soeur}(Z, T)) \wedge \text{femme}(X) \rightarrow \text{cousine}(X, Y)$$

Le code prolog est :

```
cousine(X, Y) :- parent(Z, X), frere(Z, T), parent(T, Y), X \== Y, femme(X).
cousine(X, Y) :- parent(Z, X), soeur(Z, T), parent(T, Y), X \== Y, femme(X).
```

La fin de la solitude

1. Chaque personne possède un prédicat `personne(I, N, T, C, A)` qui définit son profil. Voici un exemple de code :

```
personne(lucas, têténlaire, 1.3, brun, 13).
personne(max, ladétente, 1.5, noir, 14).
personne(barry, vatrovite, 1.2, roux, 12).
personne(caroline, lasauvage, 1.3, brun, 13).
```

```

personne(ambre, lechacal, 1.2, roux, 12).
personne(anna, ladebrouille, 1.5, noir, 14).
personne(gus, lebizarre, 1.4, blanc, 12).

```

```

gout(lucas, pop, science-fiction, jeux).
gout(max, jazz, policier, jeux).
gout(barry, rock, bd, jeux).
gout(caroline, rock, bd, jeux).
gout(ambre, jazz, policier, jeux).
gout(anna, pop, science-fiction, jeux).
gout(gus, cacophonie, fantastique, inconnu).

```

```

recherche(lucas, 1.5, noir, 14).
recherche(max, 1.2, roux, 12).
recherche(barry, 1.3, brun, 13).
recherche(caroline, 1.2, roux, 12).
recherche(ambre, 1.5, noir, 14).
recherche(anna, 1.3, brun, 13).
recherche(gus, 3.0, multicolore, 16).

```

Chaque personne a également un prédicat `gout(I, M, L, S)` qui lui est associé et qui définit ses goûts.

Enfin, chaque personne a un prédicat `recherche(I, T, C, A)` définissant le type de personne qu'elle recherche.

2. Les règles de `convient-physiquement(X, Y)` sont les suivantes :
 En français, X convient à Y si Y possède la taille T, la couleur de cheveux C et l'âge A que X recherche.
 En logique, cela s'écrit :

$$\forall X, Y, N, T, C, A \\ (X \neq Y) \wedge \text{personne}(X, N, T, C, A) \wedge \text{recherche}(Y, T, C, A) \rightarrow \text{convient} - \text{physiquement}(X, Y)$$

Le code prolog est :

```

convient-physiquement(X, Y) :- X \== Y, personne(X, _, T, C, A), recherche(Y, T, C, A).

```

Les règles de `ont-memes-gouts(X, Y)` sont les suivantes :

En français, X et Y ont les mêmes goûts s'ils aiment le même genre de musique

M, le même genre de littérature L et pratiquent le même sport S.

En logique, cela s'écrit :

$$\forall X, Y, M, L, S \\ (X \neq Y) \wedge \text{gout}(X, M, L, S) \wedge \text{gout}(Y, M, L, S) \rightarrow \text{ont} - \text{memes} - \text{gouts}(X, Y)$$

Le code prolog est :

```

ont-memes-gouts(X, Y) :- X \== Y, gout(X, M, L, S), gout(Y, M, L, S).

```

3. On en déduit que le programme qui détermine `assortis(X, Y)` sont :
En logique :

$$\begin{aligned} & \forall X, Y \\ & (X \neq Y) \wedge \text{convient} - \text{physiquement}(X, Y) \wedge \text{convient} - \text{physiquement}(Y, X) \\ & \wedge \text{ont} - \text{memes} - \text{gouts}(X, Y) \rightarrow \text{assortis}(X, Y) \end{aligned}$$

En prolog :

`assortis(X, Y) :- X \== Y, convient-physiquement(X, Y), convient-physiquement(Y, X), ont-mem`

Attention à ne pas dépasser!

1. Nous pouvons commencer à écrire le prédicat :
En français, `coloriage(C1, C2, C3, C4)` est vrai si C1, C2, C3 et C4 sont des couleurs et que C1 est différent de C2, C3 et C4 et que C3 est différent de C2 et C4.
En logique, cela s'écrit :

$$\begin{aligned} & \forall C1, C2, C3, C4 \\ & (\text{couleur}(C1) \wedge \text{couleur}(C2) \wedge \text{couleur}(C3) \wedge \text{couleur}(C4)) \\ & \wedge (C1 \neq C2) \wedge (C1 \neq C3) \wedge (C1 \neq C4) \wedge (C2 \neq C3) \wedge (C3 \neq C4) \\ & \rightarrow \text{coloriage}(C1, C2, C3, C4) \end{aligned}$$

En prolog, cela s'écrit :

`coloriage(C1, C2, C3, C4) :- couleur(C1), couleur(C2), couleur(C3), couleur(C4), C1 \== C2,`

2. En réécrivant la formule ainsi :

`coloriage(C1, C2, C3, C4) :- couleur(C1), couleur(C2), C1 \== C2, couleur(C3), C1 \== C3, C`

Je pense qu'en vérifiant plus tôt les différences entre les couleurs, cela permet au programme de passer moins d'étapes en éliminant les mauvaises combinaisons, tout de suite et de trouver une solution plus rapidement.

Graphes et récursivité

Introduction

1. Une fonction factorielle. Les règles de construction sont :

$$\begin{aligned} & \forall n, r \in \mathbb{N} \\ & \text{fact}(0, 1) \\ & (n > 0 \wedge \text{fact}(n-1, r)) \rightarrow \text{fact}(n, r * n) \end{aligned}$$

Le code prolog est :

`fact(0, 1).
fact(N, R) :- N > 0, M is N - 1, fact(M, T), R is T*N.`

2. La somme des entiers de 1 à n . Les règles de construction sont :

$$\begin{aligned} & \forall n \in \mathbb{N} \\ & \text{somme}(1, 1) \\ & (n > 1 \wedge \text{somme}(n-1)) \rightarrow \text{somme}(n) \end{aligned}$$

Le code prolog est :

```
somme(1, 1).
somme(N, R) :- N > 1, M is N-1, somme(M, T), R is T+N.
```

3. La suite de Fibonacci. Les règles de construction sont :

$$\begin{aligned} & \forall n, r, w \in \mathbb{N} \\ & \text{fibonacci}(1, 1) \\ & \text{fibonacci}(2, 1) \\ & (n > 2) \wedge \text{fibonacci}(n-1, r) \wedge \text{fibonacci}(n-2, w) \rightarrow \text{fibonacci}(n, r+w) \end{aligned}$$

Le code prolog est :

```
fibonacci(1, 1).
fibonacci(2, 1).
fibonacci(N, R) :- N > 2, fibonacci(N-1, X), fibonacci(N-2, Y), R is X+Y.
```

4. La fonction d'Ackermann; Les règles de construction sont :

$$\begin{aligned} & \forall m, n \in \mathbb{N} \\ & A(m, n) = \begin{cases} n+1 & \text{si } m = 0 \\ A(m-1, 1) & \text{si } m > 0 \text{ et } n = 0 \\ A(m-1, A(m, n-1)) & \text{si } m = 0 \text{ et } n > 0 \end{cases} \end{aligned}$$

Le code prolog est :

```
ack(M, N, T) :- M == 0, T is N+1.
ack(M, N, T) :- M > 0, N == 0, Z is M-1, ack(Z, 1, A), T is A.
ack(M, N, T) :- M > 0, N > 0, X is M-1, B is N-1, ack(M, B, Y), ack(X, Y, A), T is A.
```

Graphes dirigés acycliques

1. `chemin_oriente(X, Y)` veut dire « Il existe un chemin orienté allant de X à Y ».

Son implication est :

$$\begin{aligned} & \forall X, Y, Z \\ & \text{arete}(X, Y) \vee (\text{arete}(X, Z) \wedge \text{chemin_orienté}(Z, Y)) \rightarrow \text{chemin_orienté}(X, Y) \end{aligned}$$

Son code prolog est :

```
chemin_orienté(X, Y) :- arete(X, Y).
chemin_orienté(X, Y) :- arete(X, Z), chemin_orienté(Z, Y).
```

2. `chemin_orienté(X, Y, N)` veut dire « Il existe un chemin orienté de longueur N allant de X à Y ».

Son implication est :

$$\begin{aligned} & \forall X, Y, Z \text{ et } N \in \mathbb{N} \\ & \text{arete}(X, Y) \rightarrow \text{chemin_orienté}(X, Y, 1) \\ & (\text{arete}(X, Z) \wedge \text{chemin_orienté}(Z, Y, N)) \rightarrow \text{chemin_orienté}(X, Y, N + 1) \end{aligned}$$

Son code prolog est :

```
chemin_orienté(X, Y, 1) :- arete(X, Y).
chemin_orienté(X, Y, N) :- arete(X, Z), chemin_orienté(Z, Y, M), N is 1+M.
```

3. `chemin(X, Y)` veut dire « Il existe un chemin non orienté de longueur N allant de X à Y ».

Son implication est :

$$\begin{aligned} & \forall X, Y, Z \\ & \text{chemin_orienté}(X, Y) \wedge \text{chemin_orienté}(Y, X) \rightarrow \text{chemin}(X, Y) \end{aligned}$$

Son code prolog est :

```
chemin(X, Y) :- chemin_orienté(X, Y).
chemin(X, Y) :- chemin_orienté(Y, X).
chemin(X, Y) :- chemin_orienté(Z, X), chemin_orienté(Z, Y).
```

On peut en déduire le prédicat `connecte`. Son code prolog est :

```
connecte([]).
connecte([X]) :- chemin(X, _).
connecte([X | [Y]]) :- chemin(X, Y).
connecte([X | [Y | Q]]) :- chemin(X, Y), connecte(Q).
```

On doit inclure une liste de les sommets qu'on veut vérifier.

Listes et langages

Introduction

1. En tapant la ligne `[X | Y] = [a, b, c, d]`., on obtient :

```
X = a,
Y = [b, c, d]
```

X est la tête de la liste tandis que Y est la queue.

2. En tapant la ligne `[X] = [a, b, c, d]`, on obtient :

```
false
```

Mis ainsi, on considère que X est une variable. Mais elle ne peut pas récupérer toute les valeurs de la liste, juste a ou $[b, c, d]$. Donc, cette variable ne peut être que fausse.

Opérations sur les listes

1. Le code du prédicat `head(X, L)` est `head(X, [X | _])`.. Le premier élément est la tête de la liste.
2. Le code du prédicat `addhead(X, L, L1)` est `addhead(X, L, [X | L])`. Il s'agit de placer cet élément en tête de la liste.
3. Le code du prédicat `last(X, L)` est :

```
last(X, [X]).
last(X, [_ | L]) :- last(X, L).
```

Cette fonction récursive parcourt la liste jusqu'à qu'il n'y ait qu'un seul élément.

4. Je vois qu'en exécutant la ligne `addlast(e, [a,b,c,d], Resultat)`., on obtient `X = [[a, b, c, d] | e]`
Cela est un problème parce que la liste est divisé en deux, on a `[a, b, c, d]` comme tête et `e` comme queue alors que le résultat attendu est `[a, b, c, d, e]`. La solution que j'ai trouvée est :

```
addlast(X, [], [ X ]).
addlast(X, [Y | Q1], Q3) :- addlast(X,Q1,Q2), addhead(Y, Q2, Q3).
```

5. Le code du prédicat `reverse(L, L1)` est :

```
reverse([X], [X]).
reverse([X | L1], L2) :- reverse(L1, L3), addlast(X, L3, L2).
```

Construction inductive et langages

Voici les constructions annexes utilisées :

```
longueur([], 0).
longueur([_ | Q], N) :- longueur(Q, N2), N is N2 + 1.

nombre_de([], _, 0).
nombre_de([X | Q], A, N) :- nombre_de(Q, A, N2), A == X, N is N2 + 1.
nombre_de([X | Q], A, N) :- nombre_de(Q, A, N2), A \== X, N is N2.

pop([X], [], X).
pop([Y | Q1], Q3, X) :- pop(Q1, Q2, X), addhead(Y, Q2, Q3).

langagea([]).
langagea([a | S]) :- langagea(S).

langageb([]).
langageb([b | S]) :- langageb(S).
```

```
langagec([]).
langagec([c | S]) :- langagec(S).
```

1. Pour construire $a^n b, n \in \mathbb{N}$
La construction logique est :

$$b \in \mathcal{L}_1$$

$$S \in \mathcal{L}_1 \rightarrow aS \in \mathcal{L}_1$$

Le code prolog est :

```
langage1([b]).
langage1([a | S]) :- langage1(S).
```

2. Pour construire $ab^n, n \in \mathbb{N}$
La construction logique est :

$$a \in \mathcal{L}_2$$

$$S \in \mathcal{L}_2 \rightarrow Sb \in \mathcal{L}_2$$

Le code prolog est :

```
langagea([]).
langagea([a | S]) :- langagea(S).
```

```
langageb([]).
langageb([b | S]) :- langageb(S).
```

```
langagec([]).
langagec([c | S]) :- langagec(S).
```

```
langage2([a]).
langage2([a | S]) :- langageb(S).
```

3. Pour construire $a^n b^m, n, m \in \mathbb{N}$
La construction logique est :

$$\epsilon \in \mathcal{L}_1$$

$$S \in \mathcal{L}_3 \rightarrow aS \in \mathcal{L}_3$$

$$S \in \mathcal{L}_3 \rightarrow Sb \in \mathcal{L}_3$$

Le code prolog est :

```
langage3([]).
langage3([a | S]) :- langage3(S).
langage3([b | S]) :- langageb(S).
```

4. Pour construire $a^{2n}, n \in \mathbb{N}$
La construction logique est :

$$\begin{aligned}\epsilon &\in \mathcal{L}_4 \\ S \in \mathcal{L}_1 &\rightarrow aaS \in \mathcal{L}_4\end{aligned}$$

Le code prolog est :

```
langage4([]).
langage4([a, a | S]) :- langage4(S).
```

5. Pour construire $a^n b^n, n \in \mathbb{N}$
La construction logique est :

$$\begin{aligned}\epsilon &\in \mathcal{L}_5 \\ S \in \mathcal{L}_5 &\rightarrow aSb \in \mathcal{L}_5\end{aligned}$$

Le code prolog est :

```
langage5([]).
langage5(L) :- langage3(L), nombre_de(L, a, N1), nombre_de(L, b, N2), N1 == N2.
```

6. Pour construire l'ensemble des mots *palindromes* sur a, b . Un mot sur cet alphabet est palindrome si on peut le lire de gauche à droite ET de droite à gauche. Par exemple, *abba* et *bab* sont des palindromes mais *abab* et *bba* n'en sont pas.

La construction logique est :

$$\begin{aligned}\epsilon &\in \mathcal{L}_6 \\ S \in \mathcal{L}_6 &\rightarrow aSa \in \mathcal{L}_6 \\ S \in \mathcal{L}_6 &\rightarrow bSb \in \mathcal{L}_6\end{aligned}$$

Le code prolog est :

```
langage6([]).
langage6([_]).
langage6([X, X]).
langage6([X | Q]) :- langage6(Q2), pop(Q, Q2, Y), X == Y.
```

7. Pour construire $a^l b^m c^n, l, m, n \in \mathbb{N}$
La construction logique est :

$$\begin{aligned}\epsilon &\in \mathcal{L}_7 \\ S \in \mathcal{L}_7 &\rightarrow Sc \in \mathcal{L}_7 \\ S \in \mathcal{L}_7 &\rightarrow aS \in \mathcal{L}_7\end{aligned}$$

Le code prolog est :

```
langage7ann([]).
langage7ann([b | S]) :- langage7ann(S).
```

```

langage7ann([c | S]) :- langagec(S).
langage7([]).
langage7([a | S]) :- langage7(S).
langage7([b | S]) :- langage7ann(S).
langage7([c | S]) :- langagec(S).

```

8. Pour construire $a^m b^n c^m, m, n \in \mathbb{N}$
 La construction logique est :

$$\begin{aligned} \epsilon &\in \mathcal{L}_8 \\ S \in \mathcal{L}_8 &\rightarrow aSc \in \mathcal{L}_8 \end{aligned}$$

Le code prolog est :

```

langage8([]).
langage8(L) :- langage7(L), nombre_de(L, a, N1), nombre_de(L, c, N3), N1 == N3.

```

9. Pour construire le langage $\mathcal{L}$ qui contient tous les mots sur a, b contenant au plus un b . Par exemple $aaabaaaaaa$ et aa sont dans $\mathcal{L}$ mais pas $baaaaaaaab$.

La construction logique est :

$$\begin{aligned} \epsilon &\in \mathcal{L}_9 \\ S \in \mathcal{L}_9 &\rightarrow aS \in \mathcal{L}_9 \\ S \in \mathcal{L}_9 &\rightarrow Sa \in \mathcal{L}_9 \end{aligned}$$

Le code prolog est :

```

langage9ann([]).
langage9ann([a | L]) :- langage9(L).
langage9ann([b | L]) :- langage9ann(L).
langage9(L) :- langage9ann(L), nombre_de(L, b, N), N < 2.

```

10. Pour construire le langage $\mathcal{L}$ qui contient tous les mots ayant le même nombre de a et de b . Par exemple $aaababbb$ et $abba$ sont dans $\mathcal{L}$ mais pas $aabbb$.

$$\begin{aligned} \epsilon &\in \mathcal{L}_{10} \\ S \in \mathcal{L}_{10} &\rightarrow aSb \in \mathcal{L}_{10} \\ S \in \mathcal{L}_{10} &\rightarrow bSa \in \mathcal{L}_{10} \end{aligned}$$

Le code prolog est :

```

langage10([]).
langage10(L) :- nombre_de(L, a, N1), nombre_de(L, b, N2), N1 == N2.

```

11. Pour construire $a^n b^n c^n, n \in \mathbb{N}$
 La construction logique est :

$$\begin{aligned} \epsilon &\in \mathcal{L}_{11} \\ S \in \mathcal{L}_{11} &\rightarrow abSc \in \mathcal{L}_{11} \\ S \in \mathcal{L}_{11} &\rightarrow aSbc \in \mathcal{L}_{11} \end{aligned}$$

Le code prolog est :

```
langage11([]).  
langage11(L) :- langage7(L), nombre_de(L, a, N1), nombre_de(L, b, N2), nombre_de(L, c, N3),
```